UD5. Programación de bases de datos

Bloque 4. Procedementos almacenados e funcións de usuario

Os procedementos almacenados e as funcións permiten incorporar lóxica de programación directamente no servidor MariaDB. Mediante estes elementos é posible encapsular operacións, reutilizar código e centralizar tarefas relacionadas coa xestión da información.

1. Procedementos almacenados

1.1. Concepto

Un procedemento almacenado é un conxunto de instrucións SQL e estruturas procedementais que se gardan na base de datos para poder executalas posteriormente mediante unha chamada.

A diferenza dunha consulta SQL convencional, que se executa directamente, un procedemento pode recibir parámetros, declarar variables locais, realizar operacións, tomar decisións e repetir instrucións.

Os procedementos almacenados utilízanse para:

- Automatizar operacións habituais sobre as táboas.
- Centralizar regras de negocio no servidor.
- Realizar operacións que necesitan varias instrucións SQL.
- Validar datos antes de inserilos ou modificalos.
- Reducir a repetición de código nas aplicacións.
- Controlar as operacións que poden executar os usuarios.

1.2. Vantaxes da programación no servidor

Vantaxe	Descrición
Reutilización	Un mesmo procedemento pode executarse desde diferentes aplicacións ou sesións.
Centralización	A lóxica de negocio almacénase nun único lugar, no servidor.
Mantemento	Permite modificar unha operación sen ter que actualizar todas as aplicacións que a utilizan.
Seguridade	Pódense conceder permisos para executar procedementos sen dar acceso directo a todas as táboas implicadas.
Comunicación	Pode reducir o número de instrucións que unha aplicación necesita enviar ao servidor.

A programación no servidor tamén implica responsabilidades: é necesario controlar os permisos, documentar o código e evitar procedementos excesivamente complexos ou difíciles de manter.

1.3. Sintaxe de CREATE PROCEDURE

A instrución `CREATE PROCEDURE` permite crear un procedemento almacenado.

```
DELIMITER //

CREATE PROCEDURE nome_procedemento()
BEGIN
    -- Instrucións SQL
    -- Variables, condicións e bucles
END //

DELIMITER ;
```

Os parénteses conteñen os parámetros do procedemento. Se non recibe parámetros, déixanse baleiros.

O corpo do procedemento delimítase mediante `BEGIN` e `END`. Dentro deste bloque poden aparecer instrucións SQL e estruturas de programación.

O delimitador en MySQL Workbench e MariaDB

O cliente SQL utiliza normalmente o punto e coma (`;`) para identificar o final dunha instrución. Como un procedemento pode conter varias instrucións separadas por punto e coma, cámbiase temporalmente o delimitador mediante `DELIMITER` .

A instrución `DELIMITER` é unha directiva do cliente, non unha instrución SQL executada polo servidor. Ao rematar a creación do procedemento, adoita restaurarse o delimitador habitual.

Exemplo: crear un procedemento sinxelo

```
DELIMITER //

CREATE PROCEDURE listar_alumnos()
BEGIN
    SELECT *
    FROM alumnos;
END //

DELIMITER ;
```

Este procedemento executa unha consulta que devolve todos os rexistros da táboa `alumnos` .

1.4. Executar un procedemento mediante CALL

Para executar un procedemento almacenado utilízase a instrución `CALL` , seguida do nome do procedemento e dos seus argumentos.

```
CALL nome_procedemento();
```

No exemplo anterior, a chamada sería:

```
CALL listar_alumnos();
```

Un procedemento pode devolver resultados mediante instrucións `SELECT` , modificar datos ou proporcionar valores a través de parámetros de saída.

2. Parámetros dos procedementos

2.1. Concepto

Os parámetros permiten transmitir información ao procedemento cando se executa e, nalgúns casos, recuperar valores calculados durante a súa execución.

MariaDB permite definir tres tipos de parámetros:

- `IN` : parámetro de entrada.
- `OUT` : parámetro de saída.
- `INOUT` : parámetro de entrada e saída.

2.2. Parámetros IN

Un parámetro `IN` permite enviar un valor ao procedemento. O procedemento pode utilizalo nas súas operacións, pero as modificacións da variable correspondente non se devolven ao programa que realizou a chamada.

```
CREATE PROCEDURE nome(IN parametro TIPO)
BEGIN
    -- Utilización do parámetro
END;
```

Exemplo: consultar alumnos dun curso

```
DELIMITER //

CREATE PROCEDURE alumnos_por_curso(IN p_curso VARCHAR(10))
BEGIN
    SELECT *
    FROM alumnos
    WHERE curso = p_curso;
END //

DELIMITER ;

-- Consultar os alumnos do curso DAW1
CALL alumnos_por_curso('DAW1');
```

O procedemento recibe o nome dun curso e utiliza ese valor como condición da consulta.

2.3. Parámetros OUT

Un parámetro `OUT` permite devolver un valor ao programa que chamou ao procedemento. O seu valor inicial non se utiliza como dato de entrada.

```
CREATE PROCEDURE nome(OUT parametro TIPO)
BEGIN
    -- Asignar o valor que se quere devolver
END;
```

Para recuperar o resultado desde unha sesión SQL pódese utilizar unha variable de usuario.

Exemplo: contar alumnos

```
DELIMITER //

CREATE PROCEDURE contar_alumnos(OUT p_total INT)
BEGIN
    SELECT COUNT(*)
    INTO p_total
    FROM alumnos;
END //

DELIMITER ;

-- Chamar ao procedemento
CALL contar_alumnos(@total);

-- Consultar o valor devolto
SELECT @total AS total_alumnos;
```

O procedemento calcula o número de alumnos e almacena o resultado no parámetro de saída. A chamada utiliza a variable `@total` para recuperar o valor.

2.4. Parámetros INOUT

Un parámetro `INOUT` permite enviar un valor inicial ao procedemento e devolver o seu valor modificado.

```
CREATE PROCEDURE nome(INOUT parametro TIPO)
BEGIN
    -- Utilizar e modificar o parámetro
END;
```

Exemplo: incrementar un valor

```
DELIMITER //

CREATE PROCEDURE incrementar(INOUT p_numero INT)
BEGIN
    SET p_numero = p_numero + 1;
END //

DELIMITER ;

-- Establecer o valor inicial
SET @numero = 10;

-- Enviar o valor e recuperar o resultado
CALL incrementar(@numero);

-- Consultar o novo valor
SELECT @numero AS numero_incrementado;
```

A variable comeza co valor 10. O procedemento increméntaa e, despois da chamada, o seu valor pasa a ser 11.

2.5. Comparación dos parámetros

Tipo	Recibe datos?	Devolve datos?	Uso habitual
`IN`	Si	Non directamente	Enviar criterios ou valores ao procedemento.
`OUT`	Non	Si	Devolver resultados calculados.
`INOUT`	Si	Si	Modificar un valor recibido e devolvelo.

Importante
Para os parámetros `OUT` e `INOUT`, cando se realizan chamadas desde SQL é habitual utilizar variables de usuario, como `@resultado`. Isto permite consultar posteriormente os valores devoltos.

3. Variables locais e consultas SELECT ... INTO

3.1. Declaración con DECLARE

Os procedementos almacenados poden necesitar variables para almacenar resultados intermedios, controlar condicións ou contar iteracións.

As variables locais decláranse mediante `DECLARE` ao comezo do bloque correspondente.

```
DECLARE nome VARCHAR(50);
DECLARE idade INT DEFAULT 18;
DECLARE total DECIMAL(10,2) DEFAULT 0;
```

Pódese establecer un valor inicial mediante a cláusula `DEFAULT`.

3.2. Asignación mediante SET

A instrución `SET` permite asignar ou modificar o valor dunha variable local.

```
DECLARE contador INT DEFAULT 0;

SET contador = 10;
SET contador = contador + 1;
```

3.3. Asignación mediante SELECT ... INTO

A instrución `SELECT ... INTO` permite almacenar o resultado dunha consulta directamente nunha ou varias variables locais.

```
SELECT expresion
INTO variable
FROM táboa
WHERE condicion;
```

Exemplo: obter o número de alumnos

```
DELIMITER //

CREATE PROCEDURE obter_total_alumnos()
BEGIN
    DECLARE total INT;

    SELECT COUNT(*)
    INTO total
    FROM alumnos;

    SELECT total AS numero_alumnos;
END //

DELIMITER ;

CALL obter_total_alumnos();
```

Exemplo: almacenar varios valores

```
DELIMITER //

CREATE PROCEDURE obter_datos_curso(IN p_curso VARCHAR(10))
BEGIN
    DECLARE total_alumnos INT;
    DECLARE media_notas DECIMAL(5,2);

    SELECT
        COUNT(*),
        AVG(nota)
    INTO
        total_alumnos,
        media_notas
    FROM alumnos
    WHERE curso = p_curso;

    SELECT
        total_alumnos,
        media_notas;
END //

DELIMITER ;

CALL obter_datos_curso('DAW1');
```

3.4. Aspectos que hai que ter en conta

- As variables locais deben declararse antes das instrucións executables do bloque.
- O número de variables de destino debe coincidir co número de expresións devoltas pola consulta.
- Se unha consulta non devolve filas, as variables de destino non reciben un novo valor e pode producirse a condición `NOT FOUND`.
- Se unha consulta devolve varias filas nun `SELECT ... INTO`, prodúcese un erro.
- Para consultas que poden devolver varias filas deben utilizarse outras estruturas, como cursores.

4. Integración da lóxica de programación nos procedementos

Unha das principais vantaxes dos procedementos almacenados é que permiten combinar instrucións SQL con estruturas de control de fluxo. Isto posibilita desenvolver operacións completas dentro do servidor.

4.1. Procedementos con condicións

Exemplo: comprobar a dispoñibilidade dun produto

```
DELIMITER //

CREATE PROCEDURE comprobar_stock(IN p_id_produto INT)
BEGIN
    DECLARE unidades INT;

    SELECT stock
    INTO unidades
    FROM produtos
    WHERE id = p_id_produto;

    IF unidades > 0 THEN
        SELECT 'Produto dispoñible' AS resultado;
    ELSE
        SELECT 'Produto sen existencias' AS resultado;
    END IF;
END //

DELIMITER ;

CALL comprobar_stock(1);
```

O procedemento consulta as unidades dispoñibles e utiliza unha condición para determinar a mensaxe que debe mostrar.

Neste exemplo considérase que o identificador recibido existe na táboa. A xestión de identificadores inexistentes mediante condicións e manexadores de erros estudarase posteriormente.

4.2. Procedementos con bucles

Exemplo: calcular unha suma acumulada

```
DELIMITER //

CREATE PROCEDURE sumar_numeros(IN p_limite INT, OUT p_total INT)
BEGIN
    DECLARE contador INT DEFAULT 1;
    DECLARE acumulado INT DEFAULT 0;

    WHILE contador <= p_limite DO
        SET acumulado = acumulado + contador;
        SET contador = contador + 1;
    END WHILE;

    SET p_total = acumulado;
END //

DELIMITER ;

CALL sumar_numeros(10, @resultado);

SELECT @resultado AS suma_total;
```

O procedemento recibe un límite, utiliza un bucle para sumar os números desde 1 ata ese límite e devolve o resultado mediante un parámetro `OUT`.

4.3. Procedementos que modifican datos

Os procedementos tamén poden realizar operacións de inserción, actualización e eliminación. Isto permite encapsular operacións de negocio e controlar as condicións nas que se executan.

Exemplo: aplicar un incremento salarial

```
DELIMITER //

CREATE PROCEDURE incrementar_salario(
    IN p_id_profesor INT,
    IN p_porcentaxe DECIMAL(5,2)
)
BEGIN
    IF p_porcentaxe > 0 AND p_porcentaxe <= 20 THEN

        UPDATE profesores
        SET salario = salario + (salario * p_porcentaxe / 100)
        WHERE id = p_id_profesor;

        SELECT 'Operación realizada' AS resultado;

    ELSE
        SELECT 'Porcentaxe non válida' AS resultado;
    END IF;
END //

DELIMITER ;

CALL incrementar_salario(1, 5);
```

O procedemento valida a porcentaxe recibida e só realiza a actualización se o valor está dentro do intervalo establecido.

5. Funcións definidas polo usuario

5.1. Concepto

Unha función definida polo usuario é un programa almacenado que recibe parámetros, realiza operacións e devolve un único valor mediante a instrución `RETURN`.

As funcións permiten encapsular cálculos ou transformacións que poden reutilizarse directamente nas consultas SQL.

Exemplos de aplicación:

- Calcular o prezo final dun produto.
- Determinar a cualificación correspondente a unha nota.

- Calcular a antigüidade dun empregado.
- Transformar ou normalizar determinados valores.
- Realizar cálculos específicos dunha aplicación.

5.2. Sintaxe de CREATE FUNCTION

A instrución `CREATE FUNCTION` permite crear unha función almacenada.

```
DELIMITER //

CREATE FUNCTION nome_funcion(parametros)
RETURNS tipo_dato
[caracteristicas]
BEGIN
    -- Declaración de variables
    -- Instrucións
    RETURN valor;
END //

DELIMITER ;
```

Os elementos principais son:

- **Nome:** identificador da función.
- **Parámetros:** valores que recibe a función.
- **RETURNS:** tipo de dato que devolverá.
- **BEGIN ... END:** bloque coas instrucións.
- **RETURN:** valor que devolve a función.

> **Diferenza entre RETURN e SELECT**
> Un procedemento pode devolver un ou varios conxuntos de resultados mediante instrucións `SELECT` . Unha función debe devolver un valor do tipo indicado na cláusula `RETURNS` , mediante `RETURN` .

5.3. Crear unha función sinxela

Exemplo: calcular o dobre dun número

```
DELIMITER //

CREATE FUNCTION dobre(p_numero INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN p_numero * 2;
END //

DELIMITER ;
```

A función recibe un número enteiro e devolve o seu dobre. A característica `DETERMINISTIC` indica que, para os mesmos argumentos, a función produce o mesmo resultado.

5.4. Utilizar unha función nas consultas

A diferenza dos procedementos, as funcións poden utilizarse como parte dunha expresión SQL, sempre que o contexto permita a súa execución.

```
-- Executar a función directamente
SELECT dobre(10);

-- Utilizala nunha consulta
SELECT
    nome,
    salario,
    dobre(salario) AS salario_dobre
FROM profesores;
```

5.5. Funcións con variables locais e condicións

Exemplo: calcular o prezo con desconto

```
DELIMITER //

CREATE FUNCTION prezo_con_desconto(
    p_prezo DECIMAL(10,2),
    p_desconto DECIMAL(5,2)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE resultado DECIMAL(10,2);

    IF p_desconto < 0 OR p_desconto > 100 THEN
        SET resultado = p_prezo;
    ELSE
        SET resultado = p_prezo - (p_prezo * p_desconto / 100);
    END IF;

    RETURN ROUND(resultado, 2);
END //

DELIMITER ;

SELECT prezo_con_desconto(100, 15) AS prezo_final;
```

A función recibe un prezo e unha porcentaxe de desconto. Comproba que a porcentaxe sexa válida, calcula o resultado e devolve o importe redondeado a dous decimais.

5.6. Funcións con estruturas repetitivas

Exemplo: calcular o factorial

```
DELIMITER //

CREATE FUNCTION factorial(p_numero INT)
RETURNS BIGINT
DETERMINISTIC
BEGIN
    DECLARE contador INT DEFAULT 1;
    DECLARE resultado BIGINT DEFAULT 1;

    IF p_numero < 0 THEN
        RETURN NULL;
    END IF;

    WHILE contador <= p_numero DO
        SET resultado = resultado * contador;
        SET contador = contador + 1;
    END WHILE;

    RETURN resultado;
END //

DELIMITER ;

SELECT factorial(5) AS resultado;
```

A función utiliza un bucle para calcular o produto dos números comprendidos entre 1 e o valor recibido. Se o argumento é negativo, devolve NULL.

5.7. Características das funcións

Ao crear unha función pódense indicar características que describen o seu comportamento.

Característica	Descrición
`DETERMINISTIC`	Indica que a función produce o mesmo resultado para os mesmos argumentos.
`NOT DETERMINISTIC`	Indica que o resultado pode variar aínda que os argumentos sexan iguais.
`CONTAINS SQL`	Indica que contén instrucións SQL, pero non instrucións que lean ou modifiquen datos.
`READS SQL DATA`	Indica que a función pode ler datos mediante instrucións SQL.
`MODIFIES SQL DATA`	Indica que a función pode modificar datos mediante instrucións SQL.
`NO SQL`	Indica que non contén instrucións SQL.

Estas características describen o comportamento declarado da función. Non substitúen as restricións que MariaDB aplica ás operacións permitidas dentro dunha función almacenada.

> **Restricións das funcións almacenadas**
> As funcións non teñen a mesma liberdade que os procedementos. Por exemplo, non poden devolver conxuntos de resultados mediante un SELECT normal nin chamar a procedementos que devolvan resultados. Tampouco poden executar determinadas operacións que modifiquen o estado da transacción ou realicen control explícito de transaccións.
>
> Antes de utilizar unha función para modificar datos ou realizar operacións complexas, débense consultar as restricións da versión de MariaDB instalada.

6. Diferenzas entre procedementos e funcións

Os procedementos e as funcións son dous tipos de programas almacenados, pero teñen finalidades e formas de utilización diferentes.

Característica	Procedemento	Función
Creación	`CREATE PROCEDURE`	`CREATE FUNCTION`
Execución	`CALL`	Como parte dunha expresión SQL.
Valor de retorno	Non require un valor único de retorno.	Debe devolver un valor mediante `RETURN`.
Parámetros	`IN`, `OUT` e `INOUT`.	Parámetros de entrada.
Uso habitual	Executar operacións ou procesos completos.	Realizar cálculos e devolver valores.
Uso en SELECT	Non se chama directamente como expresión.	Pódese utilizar nunha expresión SQL.

Exemplo comparativo

```
-- Procedemento: executar unha operación
CALL contar_alumnos(@total);

-- Recuperar o resultado
SELECT @total;

-- Función: devolver un valor dentro dunha consulta
SELECT factorial(5);
```

7. Modificación e eliminación de programas almacenados

7.1. Consultar os procedementos existentes

MariaDB permite consultar os procedementos almacenados definidos nunha base de datos mediante `SHOW PROCEDURE STATUS`.

```
-- Consultar os procedementos
SHOW PROCEDURE STATUS;

-- Filtrar os procedementos dunha base de datos
SHOW PROCEDURE STATUS
WHERE Db = 'centro';
```

7.2. Consultar a definición dun procedemento

A instrución `SHOW CREATE PROCEDURE` permite consultar a definición completa dun procedemento almacenado.

```
SHOW CREATE PROCEDURE listar_alumnos;
```

O resultado mostra a instrución utilizada para crear o procedemento, incluíndo os seus parámetros e o corpo do programa.

7.3. Consultar as funcións existentes

Para consultar as funcións almacenadas pódese utilizar `SHOW FUNCTION STATUS`.

```
-- Consultar as funcións
SHOW FUNCTION STATUS;

-- Filtrar as funcións dunha base de datos
SHOW FUNCTION STATUS
WHERE Db = 'centro';
```

7.4. Consultar a definición dunha función

A instrución `SHOW CREATE FUNCTION` permite consultar a definición dunha función almacenada.

```
SHOW CREATE FUNCTION factorial;
```

7.5. Modificar un procedemento ou unha función

En MariaDB, a instrución `ALTER PROCEDURE` permite modificar determinadas características dun procedemento, pero non o seu corpo ou a súa lista de parámetros.

Para cambiar a lóxica dun procedemento, o método habitual é eliminalo e crealo de novo coa definición actualizada.

```
DROP PROCEDURE IF EXISTS listar_alumnos;

DELIMITER //

CREATE PROCEDURE listar_alumnos()
BEGIN
    SELECT id, nome, curso
    FROM alumnos
    ORDER BY nome;
END //

DELIMITER ;
```

O mesmo criterio se aplica ás funcións: para modificar o seu corpo ou os seus parámetros, elimínanse e créanse de novo. `ALTER FUNCTION` permite modificar determinadas características, pero non substitúe a definición completa.

7.6. Eliminar procedementos

A instrución `DROP PROCEDURE` elimina un procedemento almacenado.

```
DROP PROCEDURE listar_alumnos;
```

Para evitar erros cando o procedemento non existe, pódese utilizar `IF EXISTS`.

```
DROP PROCEDURE IF EXISTS listar_alumnos;
```

7.7. Eliminar funcións

A instrución `DROP FUNCTION` elimina unha función almacenada.

```
DROP FUNCTION factorial;
```

Tamén se pode utilizar `IF EXISTS`:

```
DROP FUNCTION IF EXISTS factorial;
```

> **Precaución ao eliminar programas almacenados**
> Antes de eliminar un procedemento ou unha función, convén comprobar se outras aplicacións, procedementos, funcións ou eventos dependen del. A eliminación pode provocar que esas chamadas deixen de funcionar.

8. Boas prácticas

- Utilizar nomes descritivos para os procedementos, funcións e parámetros.
- Definir parámetros de entrada, saída e entrada/saída segundo a finalidade de cada operación.
- Declarar as variables locais ao comezo dos bloques.
- Utilizar `SELECT ... INTO` cando se necesite almacenar o resultado dunha consulta nunha variable.
- Separar as operacións complexas en procedementos ou funcións máis pequenos cando resulte conveniente.
- Validar os parámetros antes de realizar modificacións sobre as táboas.
- Utilizar funcións para cálculos que deban devolver un único valor e procedementos para executar procesos.
- Documentar a finalidade dos programas e dos seus parámetros.
- Consultar a definición almacenada antes de modificar ou eliminar un programa.
- Probar os procedementos e funcións con valores normais, valores límite e datos non válidos.

9. Exercicios prácticos

Os seguintes exercicios permiten practicar a creación, execución e mantemento de procedementos almacenados e funcións definidas polo usuario en MariaDB.

Exercicio 1. Creación de procedementos

1. Crea un procedemento que mostre todos os rexistros dunha táboa.
2. Define un procedemento que consulte os alumnos dun curso determinado.
3. Crea un procedemento que reciba o identificador dun alumno e mostre os seus datos.
4. Executa os procedementos mediante `CALL`.
5. Consulta as definicións dos procedementos creados.

Exercicio 2. Parámetros IN, OUT e INOUT

1. Crea un procedemento que reciba un identificador mediante IN e consulte os datos correspondentes.
2. Define un procedemento que devolva mediante OUT o número total de alumnos.
3. Crea un procedemento que devolva a media das notas dunha táboa mediante OUT.
4. Define un procedemento que reciba un número mediante INOUT e o modifique.
5. Realiza as chamadas utilizando variables de usuario e comproba os resultados.

Exercicio 3. Variables locais e SELECT ... INTO

1. Crea un procedemento que almacene nunha variable o número de rexistros dunha táboa.
2. Obtén mediante SELECT ... INTO o prezo dun produto a partir do seu identificador.
3. Almacena varias estatísticas dunha táboa en diferentes variables locais.
4. Realiza operacións matemáticas cos valores obtidos.
5. Mostra os resultados mediante unha consulta SELECT.

Exercicio 4. Procedementos con lóxica de programación

1. Crea un procedemento que valide unha nota e devolva a súa cualificación.
2. Define un procedemento que calcule unha suma acumulada utilizando un bucle.
3. Desenvolve un procedemento que actualice o prezo dun produto segundo unha porcentaxe recibida.
4. Incorpora condicións para evitar operacións con parámetros non válidos.
5. Utiliza parámetros OUT para devolver información sobre os resultados.

Exercicio 5. Creación de funcións

1. Crea unha función que calcule o dobre dun número.
2. Define unha función que calcule o prezo dun produto aplicando un desconto.
3. Crea unha función que clasifique unha nota e devolva a cualificación correspondente.
4. Implementa unha función que calcule o factorial dun número.
5. Utiliza as funcións creadas dentro de consultas SELECT.

Exercicio 6. Mantemento de procedementos e funcións

1. Consulta os procedementos e funcións existentes na base de datos.
2. Utiliza SHOW CREATE PROCEDURE para consultar unha definición.
3. Utiliza SHOW CREATE FUNCTION para consultar unha función.

4. Modifica a lóxica dun procedemento mediante a súa eliminación e creación de novo.
5. Realiza a mesma operación cunha función.
6. Elimina os programas almacenados de proba utilizando IF EXISTS.

Exercicio 7. Práctica integradora

Desenvolve un conxunto de procedementos e funcións que permitan automatizar operacións sobre unha base de datos utilizada nas prácticas.

O desenvolvemento deberá incluír:

- Polo menos dous procedementos almacenados.
- Un procedemento con parámetros IN e outro con parámetros OUT ou INOUT.
- Uso de variables locais e consultas SELECT ... INTO.
- Estruturas condicionais e repetitivas.
- Polo menos dúas funcións definidas polo usuario.
- Chamadas e consultas que permitan comprobar o funcionamento.
- Consulta das definicións e eliminación dos elementos de proba.

10. Resumo do bloque

Elemento	Finalidade	Instrucións principais
Procedemento almacenado	Executar un conxunto de instrucións SQL.	`CREATE PROCEDURE`, `CALL`
Parámetro IN	Recibir datos.	`IN`
Parámetro OUT	Devolver datos.	`OUT`
Parámetro INOUT	Recibir e devolver un valor modificado.	`INOUT`
Variables locais	Almacenar valores temporais dentro dun bloque.	`DECLARE`, `SET`
Consulta con asignación	Gardar resultados dunha consulta en variables.	`SELECT ... INTO`
Función de usuario	Realizar un cálculo e devolver un valor.	`CREATE FUNCTION`, `RETURNS`, `RETURN`
Consultar definición	Inspeccionar o código almacenado.	`SHOW CREATE PROCEDURE`, `SHOW CREATE FUNCTION`
Eliminar programas	Eliminar procedementos ou funcións.	`DROP PROCEDURE`, `DROP FUNCTION`

> **Obxectivo alcanzado**
> Ao rematar este bloque, o alumnado debe ser capaz de crear e executar procedementos almacenados e funcións de usuario en MariaDB, utilizar parámetros e variables locais, incorporar estruturas de control de fluxo e realizar operacións de consulta, cálculo e modificación de datos desde programas almacenados no servidor.